Chapter Three
Test Techniques

White box testing

- White box testing is a strategy in which testing is based on:
  - the internal paths, structure, and implementation of the software under test (SUT).
- Unlike its complement, black box testing, white box testing generally requires detailed programming skills

White box testing

- The general white box testing process is:
  - The software's implementation is analyzed.
  - Paths through the software are identified.

- Inputs are chosen to cause the software to execute selected paths. This is called path **sensitization**.
- Expected results for those inputs are determined.
- The tests are run.

- Actual outputs are compared with the expected outputs.
- A determination is made as to the proper functioning of the software.

- White box testing can be applied at all levels of system development
  - unit,
  - integration, and
  - system.

- White box testing is more than code testing—it is **path** testing.
  - Generally, the paths that are tested are within a module (unit testing).
- But we can apply the same techniques to test paths between modules within subsystems, between subsystems within systems, and even between entire systems.

Disadvantages

- The number of execution paths may be so large than they cannot all be tested.
- The test cases chosen may not detect data sensitivity errors. For example:
  - p=q/r;
  - may execute correctly except when r=0.
  - y=2*x // should read $y=x^2$
  - will pass for test cases x=0, y=0 and x=2, y=4

- White box testing assumes the control flow is correct (or very close to correct).
- Since the tests are based on the existing paths, nonexistent paths cannot be discovered through white box testing.

- The tester must have the programming skills to understand and evaluate the software under test.

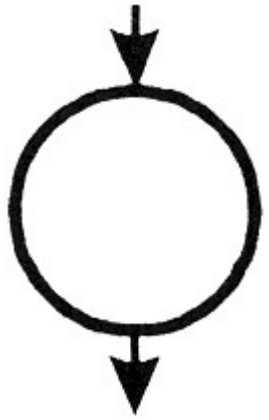
Control flow testing

- Control flow testing is one of two white box testing techniques.
- This testing approach identifies the execution paths through a module of program code and then creates and executes test cases to cover those paths.

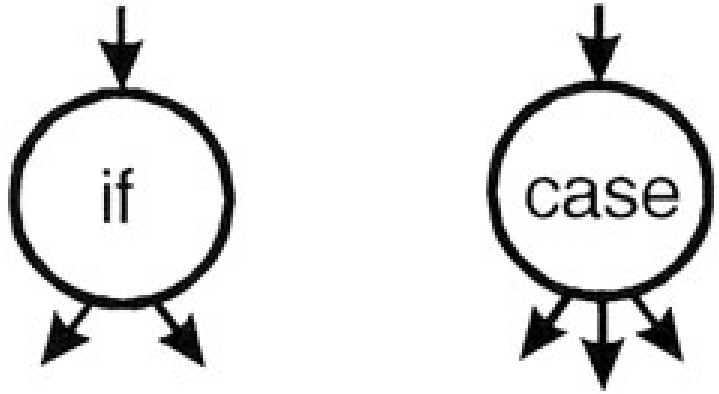
Control Flow Graphs

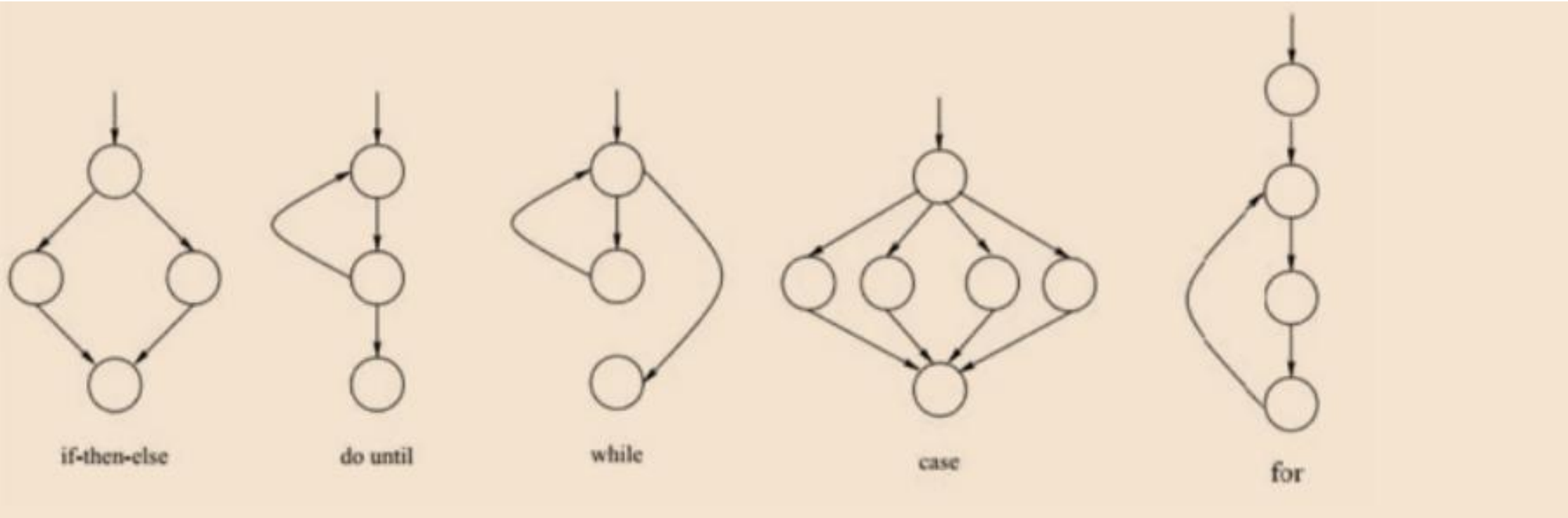
- Control flow graphs are the foundation of control flow testing.
- These graphs document the module's control structure.
- Modules of code are converted to graphs, the paths through the graphs are analyzed, and test cases are created from that analysis.
- Control flow graphs consist of a number of elements

- **Process block**
  - A process block is a sequence of program statements that execute sequentially from beginning to end.
  - No entry into the block is permitted except at the beginning.
  - No exit from the block is permitted except at the end.
  - Once the block is initiated, every statement within it will be executed sequentially.

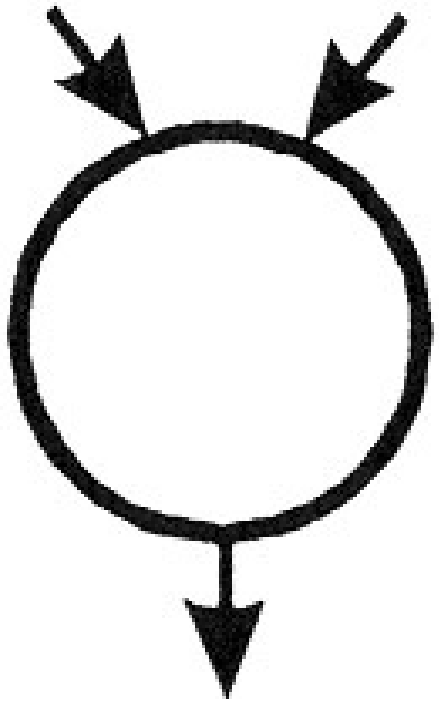


- **Decision Point**
  - A decision point is a point in the module at which the control flow can change.
  - Most decision points are binary and are implemented by if-then-else statements.
  - Multi-way decision points are implemented by case statements.
  - They are represented by a bubble with one entry and multiple exits.



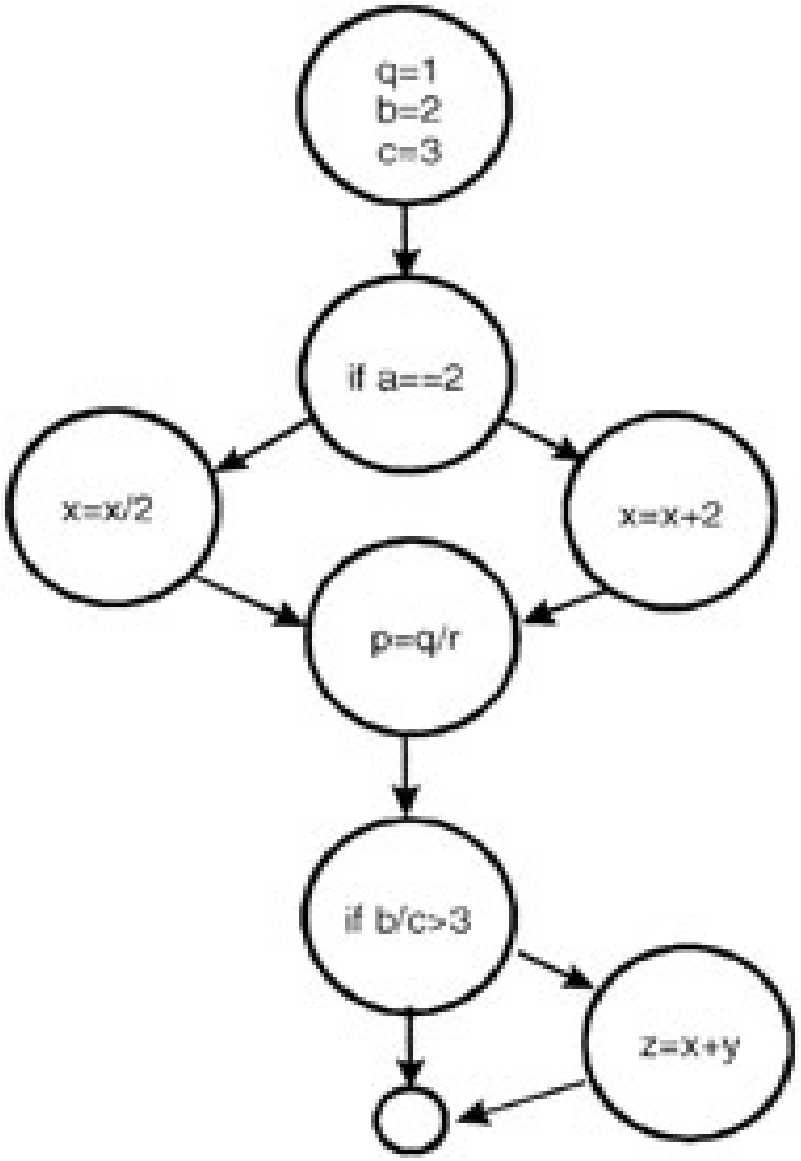

if-then-else
do until
while
case
for

- **Junction Point**
  - A junction point is a point at which control flows join together.



Example

```
q=1;
b=2;
c=3;
if (a==2) {x=x+2;}
else {x=x/2;}
p=q/r;
if (b/c>3) {z=x+y;}
```



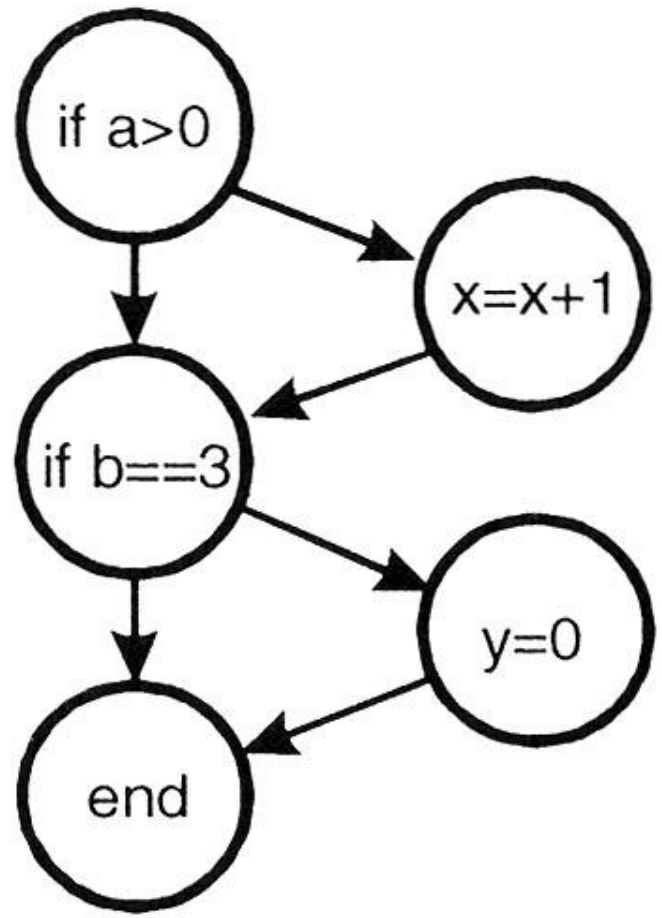
- In control flow testing we define coverage at a number of different levels.
- **Level 1**
  - The lowest coverage level is "100% statement coverage“ or statement coverage.
  - This means that every statement within the module is executed, under test, at least once.

- While this may seem like a reasonable goal, many defects may be missed with this level of coverage.
- Consider the following code snippet:

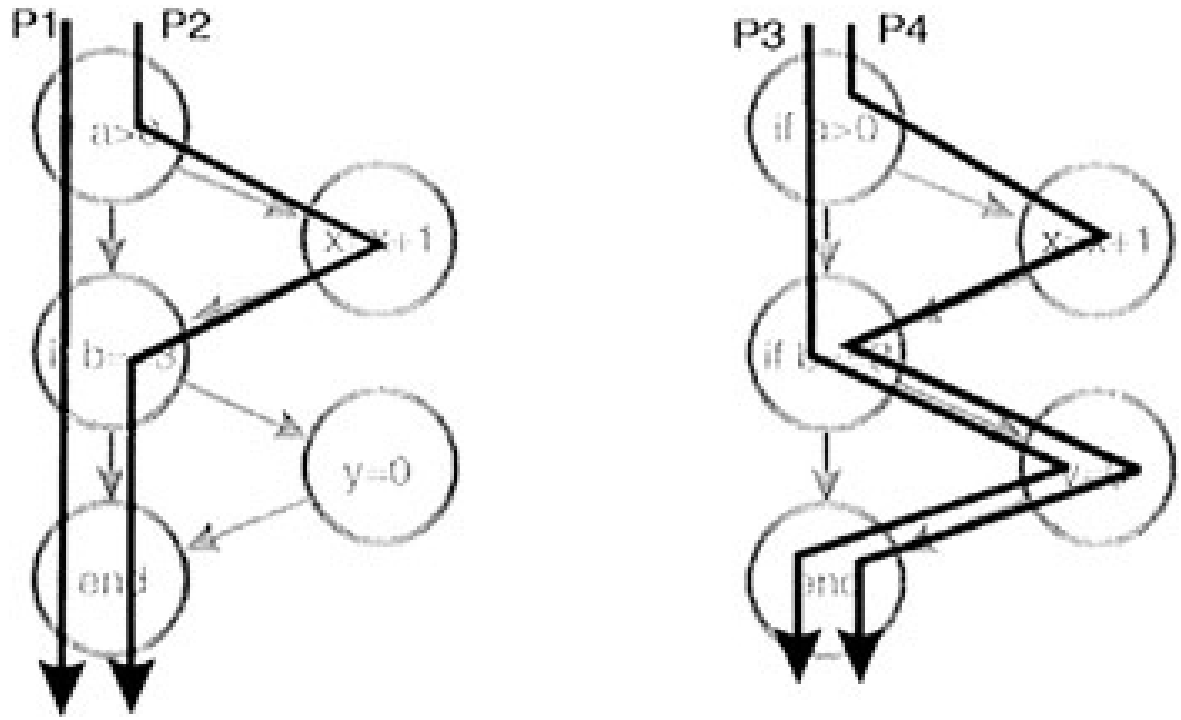
```
if (a>0)
{
        x=x+1;
}
if (b==3)
{
        y=0;
}
```

- This code can be represented in graphical form as:

Graphical representation of the above code



- **Four execution paths.**



- While a single test case is sufficient to test every line of code in this module (for example, use a=6 and b=3 as input), it is apparent that this level of coverage will miss testing many paths.
- Thus, statement coverage, while a beginning, is generally not an acceptable level of testing.

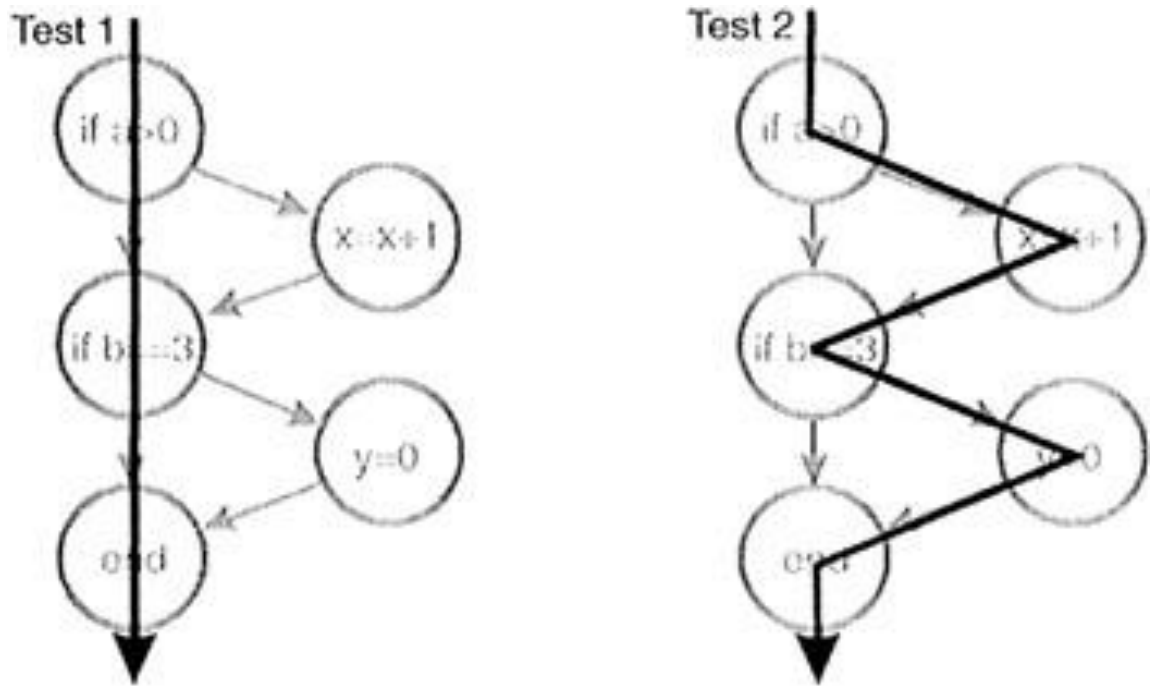
- Even though statement coverage is the lowest level of coverage, even that may be difficult to achieve in practice.
  - Often modules have code that is executed only in exceptional circumstances—low memory, full disk, unreadable files, lost connections, etc.
- Testers may find it difficult or even impossible to simulate these circumstances and thus code that deals with these problems will remain untested.

- (Statement coverage = No of statements Executed/Total no of statements in the source code * 100)
- **Example:**

```
Read A
Read B
if A > B
  Print “A is greater than B”
else
  Print "B is greater than A"
endif
```

- Set 1: If A = 5, B = 2
  - No of statements Executed: 5
  - Total no of statements in the source code: 67
  - Statement coverage =5/6*100 = 83.30 %
- Set 2: If A = 2, B = 5
  - No of statements Executed: 5
  - Total no of statements in the source code: 6
  - Statement coverage =6/6*100 = 83.30 %

- **Level 2**
  - The next level of control flow coverage is **"100% decision coverage.“ or "branch coverage."**
  - At this level enough test cases are written so that each decision that has a TRUE and FALSE outcome is evaluated at least once.
  - In the previous example this can be achieved with two test cases (a=-2, b=2 and a=4, b=3).


Test 1
x:=x+1
y:=0
Test 2

- Decision coverage= number of decision outcomes exercised/total number of decision outcomes *100
- Example

```
Example (int x)
{
   if (x < 10)
      print("x < 10")

   else
      print("x >= 10")
}
```

- For X=7
  - *Coverage*=1/2∗100=50%
- For x=13
  - *Coverage*=1/2∗100=50%

- **Level 3**
  - It is called **100% condition coverage or condition coverage**
  - At this level enough test cases are written so that each condition that has a TRUE and FALSE outcome that makes up a decision is evaluated at least once.
  - Example :

    if (a>0 && c==1) {x=x+1;}

    if (b==3 || d<0) {y=0;}
  - This level of coverage can be achieved with two test cases (a>0, c=1, b=3, d<0 and a≤0, c≠1, b≠3, d≥0).
  - **Condition coverage** is usually better than decision coverage because every individual condition is tested at least once while decision coverage can be achieved without testing every condition.

Condition Vs branch coverage

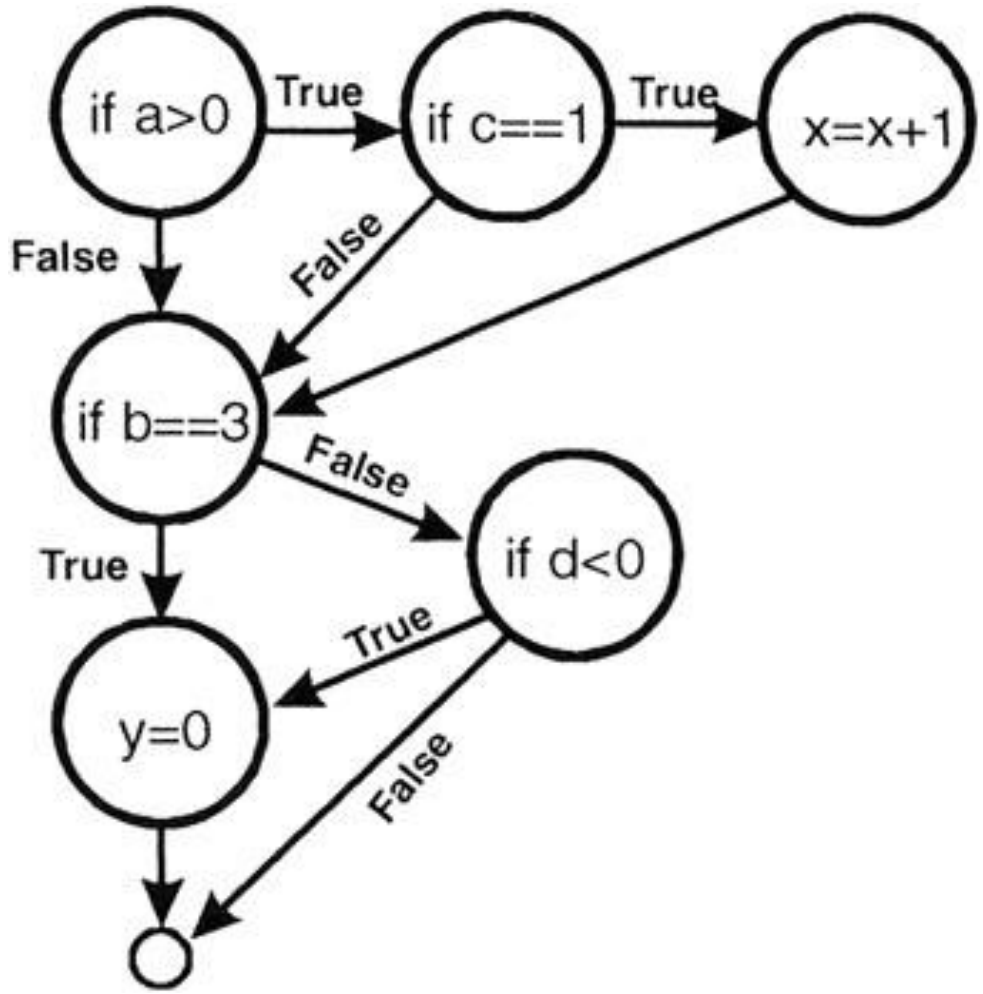
- if (A > 10 and B > 5):
  print("Valid")
  else:
  print("Invalid")

Test Case	A	B	Decision Result
1	15	10	True
2	5	10	False

- Condition B > 5 was never false in the tests. This means not all logical paths inside the decision were explored.

- **Level 4- multiple condition coverage.**
  - Consider how the programming language compiler actually evaluates the multiple conditions in a decision.
  - Use that knowledge to create test cases yielding 100% multiple condition coverage.

```
if (a>0 && c==1) {x=x+1;}
if (b==3 || d<0) {y=0;}
```



- This level of coverage can be achieved with four test cases:
  - $a>0, c=1, b=3, d<0$
  - $a\leq 0, c=1, b=3, d\geq 0$
  - $a>0, c\neq 1, b\neq 3, d<0$
  - $a\leq 0, c\neq 1, b\neq 3, d\geq 0$
- Achieving 100% multiple condition coverage also achieves decision coverage, condition coverage, and decision/condition coverage

Exercise

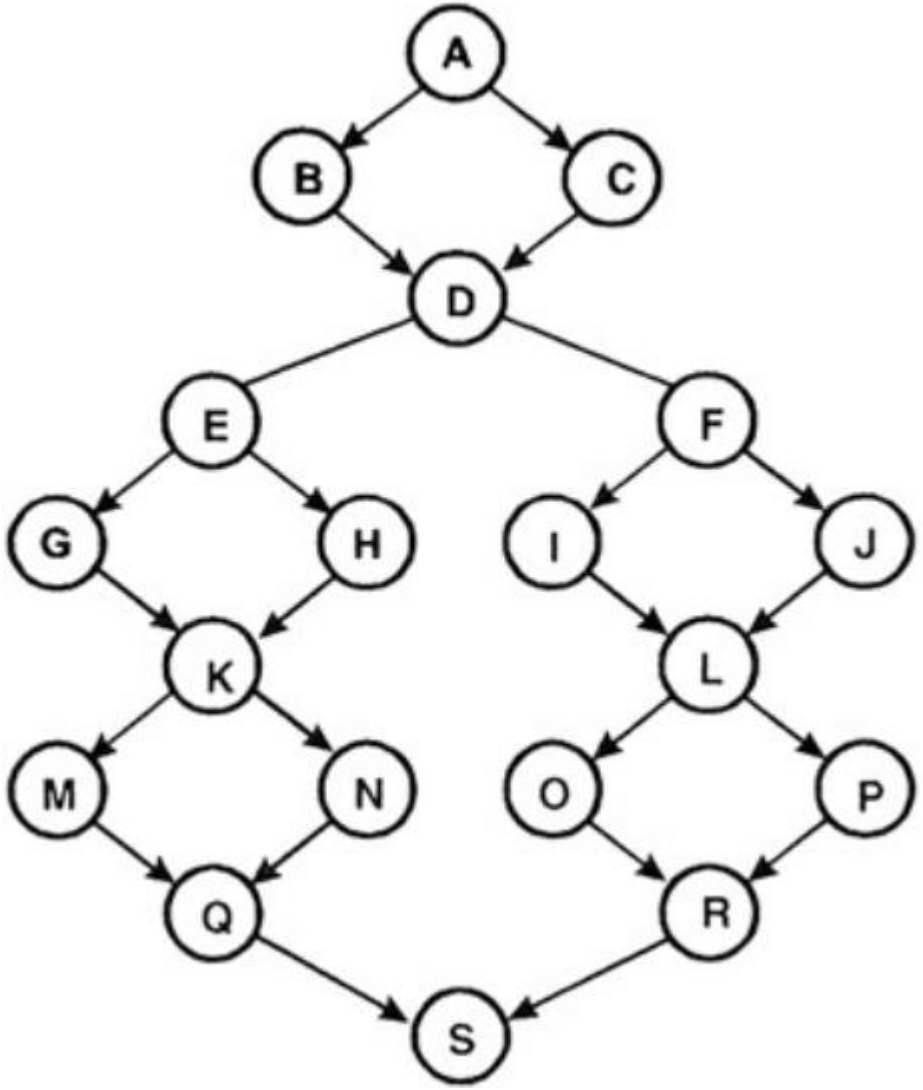
```
public static void computeStats (int [ ] numbers)
{
int length = numbers.length;
double med, var, sd, mean, sum, varsum;
sum = 0;
for (int i = 0; i < length; i++)
{
sum += numbers [ i ];
}
med = numbers [ length / 2 ];
mean = sum / (double) length;
varsum = 0;
for (int i = 0; i < length; i++)
{
varsum = varsum + ((numbers [ I ] - mean) * (numbers [ I ] - mean));
}
var = varsum / ( length - 1.0 );
sd = Math.sqrt ( var );
System.out.println ("length: " + length);
System.out.println ("mean: " + mean);
System.out.println ("median: " + med);
System.out.println ("variance: " + var);
System.out.println ("standard deviation: " + sd);
}
```

- By referring the above code
  - Draw the control flow graph
  - Select inputs that help perform
    - Level 1, 2,3 and 4 control flow testing.

Structured Testing / Basis Path Testing

- It uses an analysis of the topology of the control flow graph to identify test cases.
- The structured testing process consists of the following steps:
  - Derive the control flow graph from the software module.
  - Compute the graph's Cyclomatic Complexity (C).
  - Select a set of C basis paths.
  - Create a test case for each basis path.
  - Execute these tests.

Example

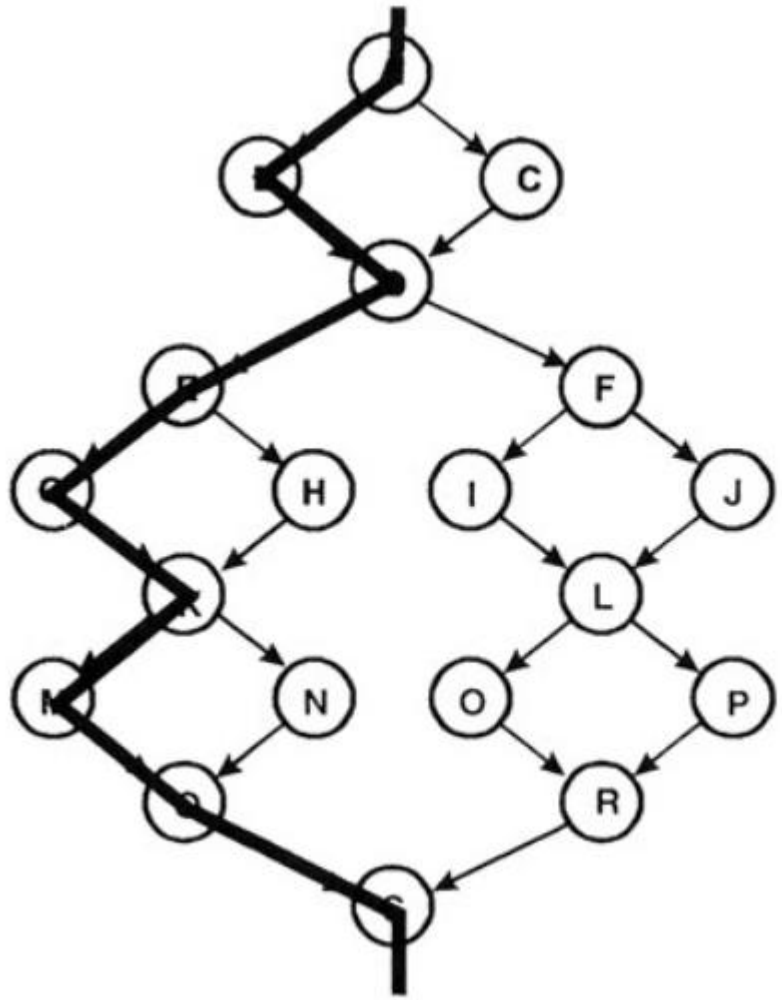


- The Cyclomatic Complexity (C) of a graph as
  - C = edges - nodes + 2(p) or
  - C=Number of decision points+1
  - C=24-19+2 = 7 or C=6+1--- 7 is the number of decision points

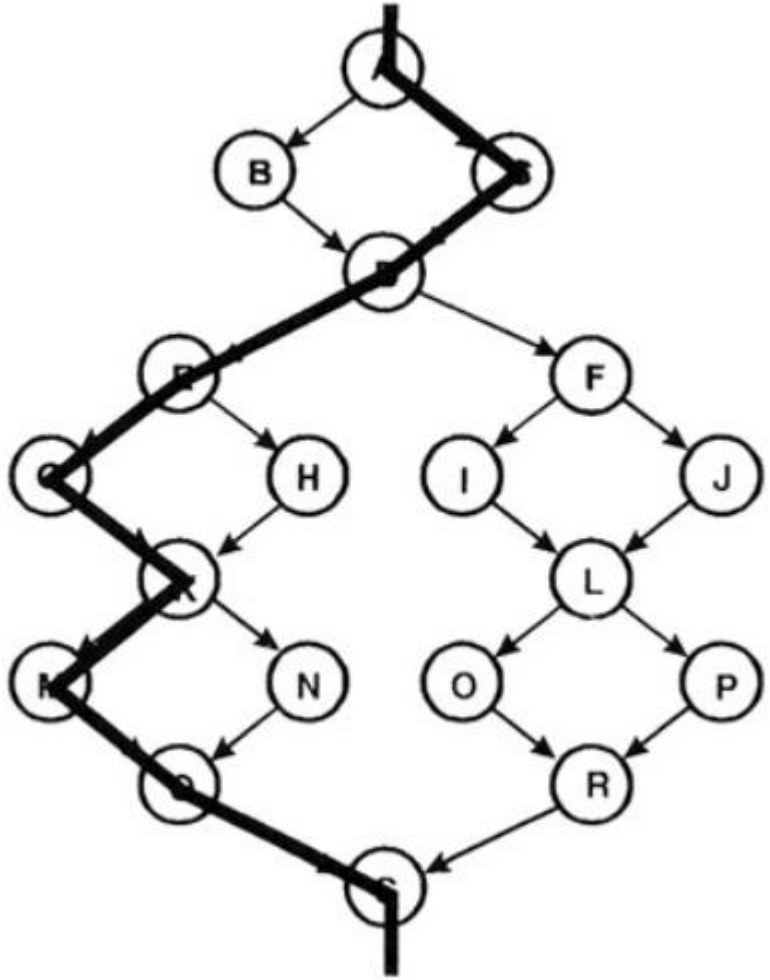
E = number of edges
N = number of nodes
P = number of connected components (usually 1 for a single program)

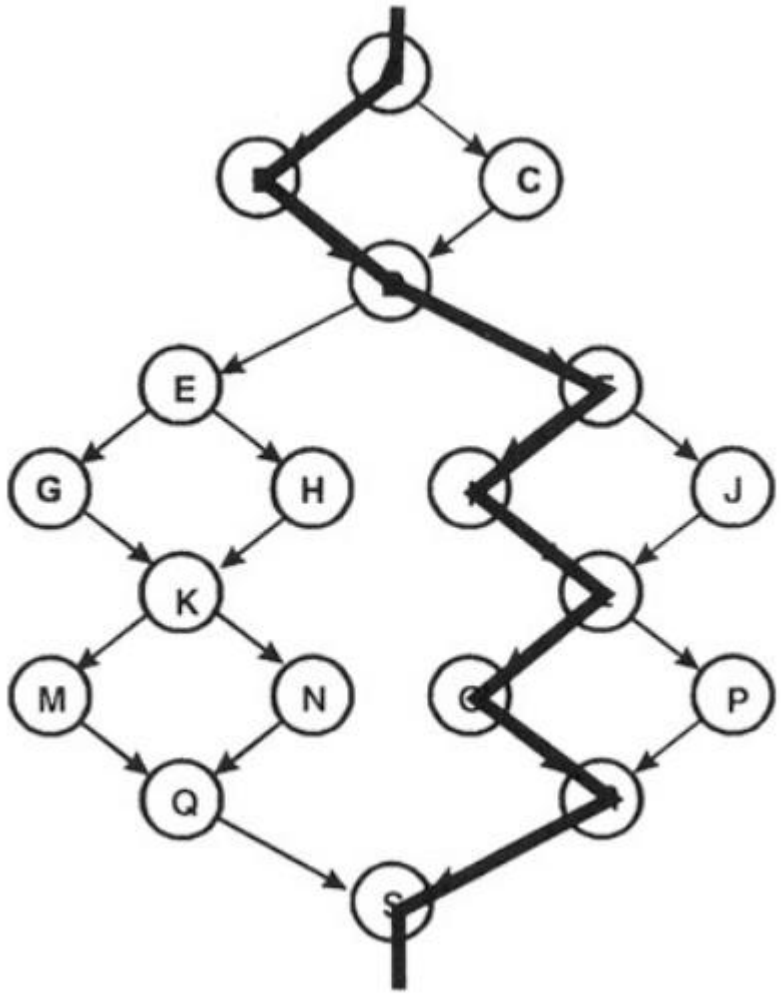
- Cyclomatic Complexity is exactly the minimum number of independent, non-looping paths (called basis paths) that can, in linear combination, generate all possible paths through the module.
- In terms of a flow graph, each basis path traverses at least one edge that no other path does.
- **Creating and executing C test cases, based on the basis paths, guarantees both branch and statement coverage**

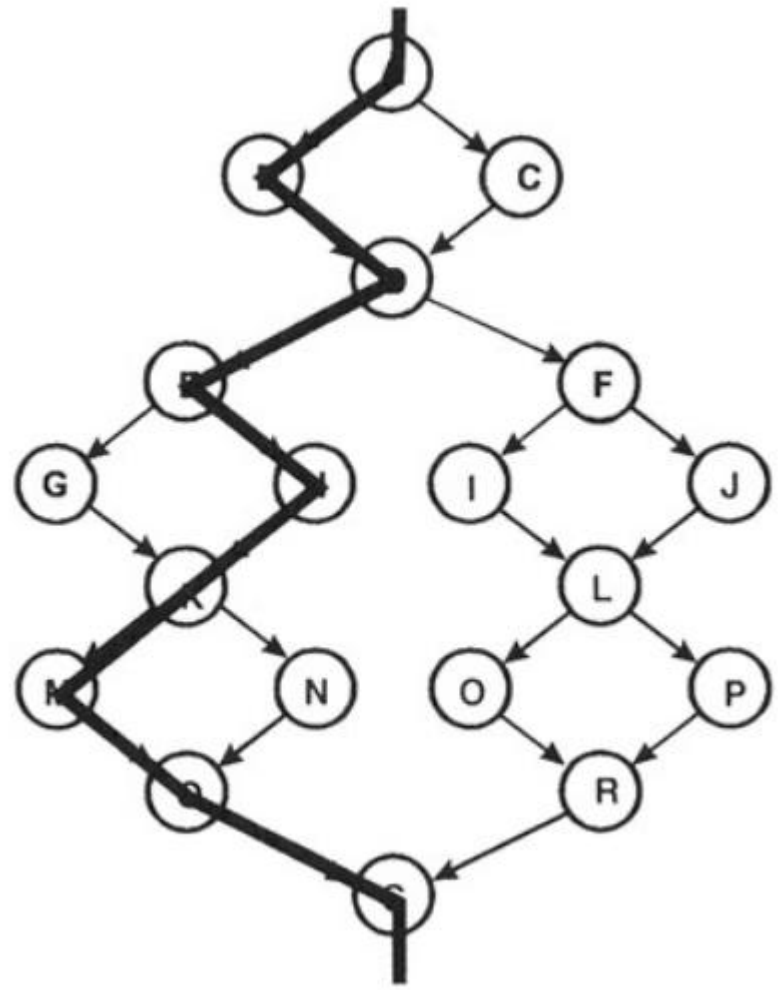
- A process for creating a set of basis paths is given by McCabe:
  - Pick a "**baseline**" path. This path should be a reasonably "typical" path of execution rather than an exception processing path.
    - **The best choice would be the most important path from the tester's view**

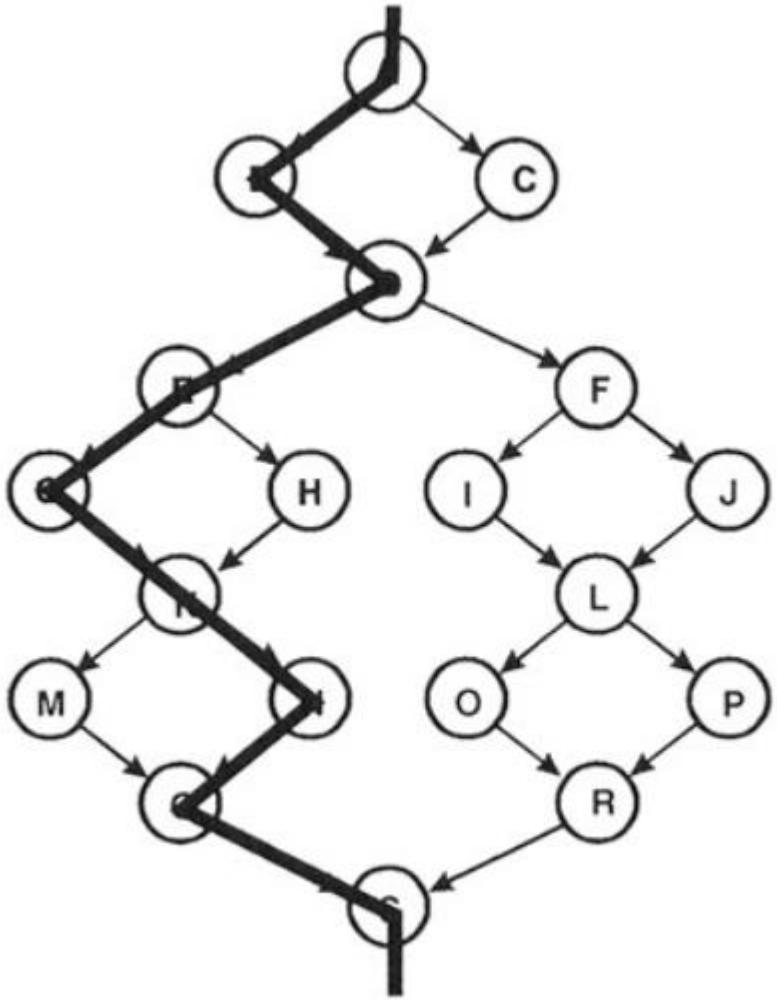

C
F
H
I
J
L
N
O
P
R

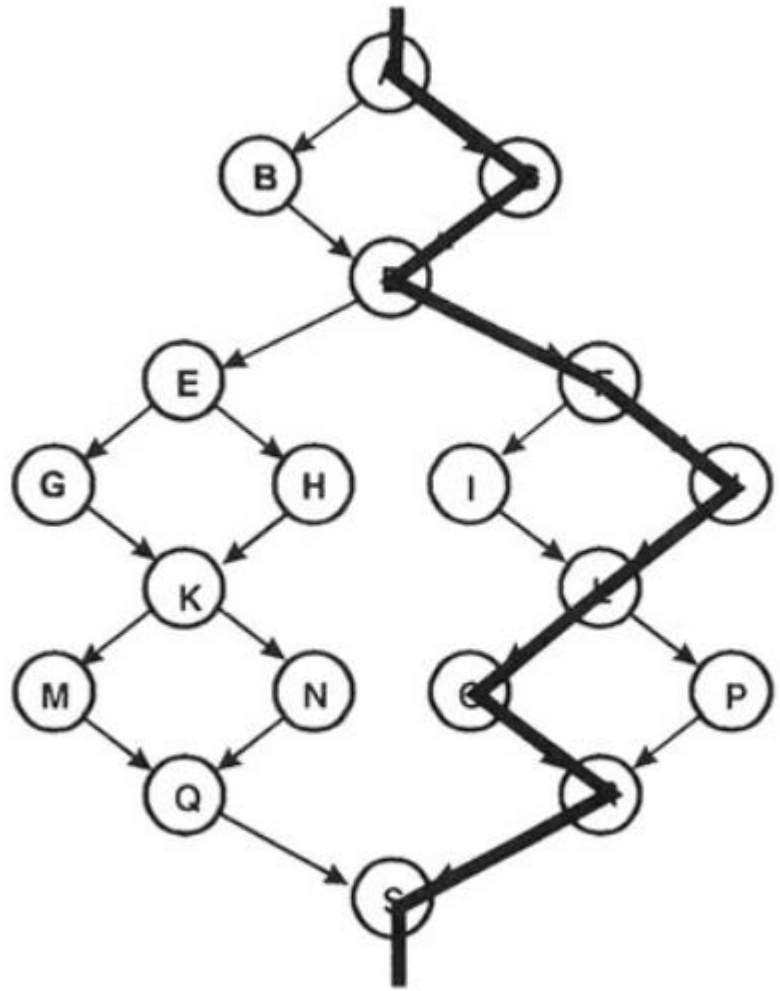
- To choose the next path, change the outcome of the first decision along the baseline path while keeping the maximum number of other decisions the same as the baseline path.

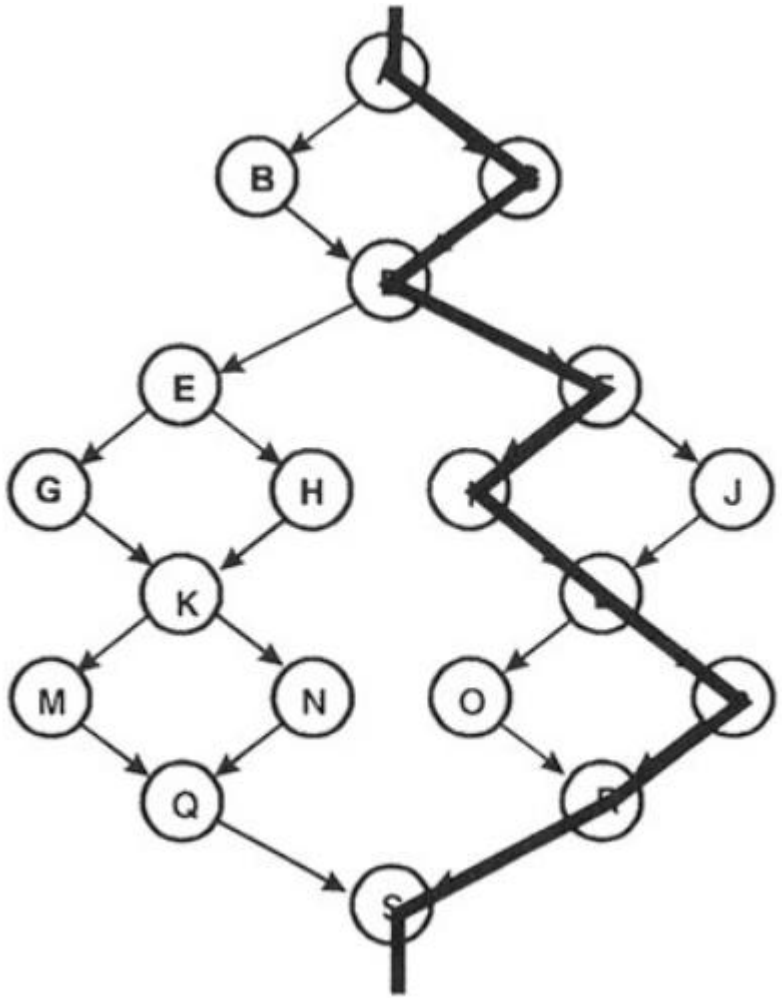



C
E
G
H
J
K
M
N
P
Q


C
F
G
I
J
L
N
O
P
R


C
F
H
I
J
L
M
O
P
R


B
E
G
H
I
K
M
N
P
Q



Thus, a set of basis paths for this graph are:

ABDEGKMQS

ACDEGKMQS

ABDFILORS

ABDEHKMQS

ABDEGKNQS

ACDFJLORS

ACDFILPRS

Structured testing calls for the creation of a test case for each of these paths. This set of test cases will guarantee both statement and branch coverage.

Exercise -1

- Below is a brief program listing. Create the control flow diagram, determine its Cyclomatic Complexity, choose a set of basis paths, and determine the necessary values for the conditions to sensitize each path.

```
if (c1) {
while (c2) {
if (c3) { s1; s2;
if (c5) s5;
else s6;
break; // Skip to end of while
}
else
if (c4) { }
else { s3; s4; break;}
} // End of while
} // End of if
s7;
if (c6) s8; s9;
s10;
```

Exercise -2

```
boolean evaluateBuySell (TickerSymbol ts) {
    s1;
    s2;
    s3;
    if (c1) {s4; s5; s6;}
    else {s7; s8;}
    while (c2) {
        s9;
        s10;
        switch (c3) {
            case-A:
                s20;
                s21;
                s22;
                break; // End of Case-A
            case-B:
                s30;
                s31;
                if (c4) {
                    s32;
                    s33;
                    s34;
                }
                else {
                    s35;
                }
                break; // End of Case-B
            case-C:
                s40;
                s41;
                break; // End of Case-C
```

```
        case-D:

            s50;
            break; // End of Case-D
          } // End Switch
        s60;
        s61;
        s62;
        if (c5) {s70; s71; }
        s80;
        s81;
} // End While
s90;
s91;
s92;
return result;
```

Data Flow Testing

- Data flow testing is a powerful tool to detect improper use of data values due to coding errors.

```
main() {
        int x;
        if (x==42){ ...}
        }
```

referencing the value of a variable without first assigning a value to it.

- Variables that contain data values have a defined life cycle. They are created, they are used, and they are killed (destroyed).
- In some programming languages creation and destruction are automatic.
- A variable is created the first time it is assigned a value and destroyed when the program exits.
- In other languages (like C, C++, and Java) the creation is formal.
- Variables are declared by statements such as:
  - int x; // x is created as an integer
  - string y; // y is created as a string

- Three possibilities exist for the first occurrence of a variable through a program path:
  - 1.~d the variable does not exist (indicated by the ~), then it is defined (d)
  - 2.~u the variable does not exist, then it is used (u)
  - 3.~k the variable does not exist, then it is killed or destroyed (k)

- The first is correct.
  - The variable does not exist and then it is defined.
- The second is incorrect.
  - A variable must not be used before it is defined.
- The third is probably incorrect.
  - Destroying variable before it is created is indicative of a programming error.

- **Defined (d)**: A variable receives a value.
  Examples: assignment, initialization, function parameter passing, reading input.
- **Used (u)**: A variable's value is read.
  Two types:
  - **Computational Use (c-use)**: used in calculations, e.g., y = x + 1.
  - **Predicate Use (p-use)**: used in a condition, e.g., if x > 0.
- **Killed (k)**: A variable becomes inaccessible or its value is destroyed.
  Examples: exiting the scope where the variable was declared, freeing memory, or the program ends.

- Examine time-sequenced pairs of defined, used, and killed variable references.

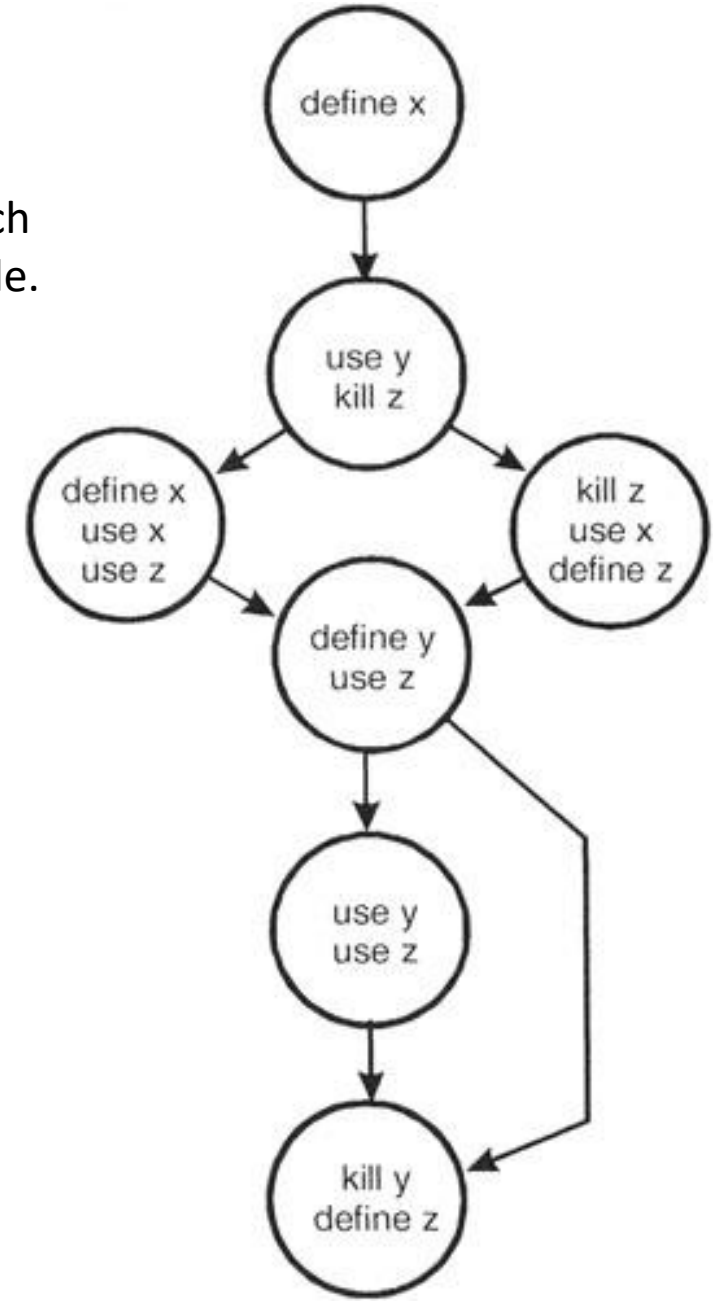
Dd-Defined and defined again	not invalid but suspicious	Probably a programming error
Du- Defined and used	perfectly correct	The normal case
Dk- Defined and then killed	not invalid	probably a programming error
ud Used and defined	acceptable	
uu Used and used again	acceptable	
uk Used and killed	acceptable	
kd Killed and defined	acceptable	A variable is killed and then redefined.

KU- Killed and used	a serious defect	Using a variable that does not exist or is undefined is always an error
kk Killed and killed	Invalid	probably a programming error

- A data flow graph is similar to a control flow graph in that it shows the data flow through a module.
- It is done in two ways:
  - Static Data Flow Testing
  - Dynamic Data Flow Testing

Static Data Flow Testing

Define-use-kill information for each of the variables used in the module.



- The define-use-kill patterns for x (taken in pairs as we follow the paths) are:
  - ~define correct, the normal case
  - define-define suspicious, perhaps a programming error
  - define-use correct, the normal case

- The define-use-kill patterns for y (taken in pairs as we follow the paths) are:
  - ~use major blunder
  - use-define acceptable
  - define-use correct, the normal case
  - use-kill acceptable
  - define-kill probable programming error

- The define-use-kill patterns (taken in pairs as we follow the paths) are:
  - ~kill programming error
  - kill-use major blunder
  - use-use correct, the normal case
  - use-define acceptable
  - kill-kill probably a programming error
  - kill-define acceptable
  - define-use correct, the normal case

- In performing a static analysis on this data flow model the following problems have been discovered:
  - x: define-define
  - y: ~use
  - y: define-kill
  - z: ~kill
  - z: kill-use
  - z: kill-kill

```
int factorial (int n) {
int answer, counter;
answer = 1;
counter = 1;
loop:
if (counter > n) return answer;
answer = answer * counter;
counter = counter + 1;
goto loop;
}
```

Drawback

- Arrays are defined and destroyed as a unit but specific elements of the array are used individually. In the general case, static analysis cannot determine whether the define-use-kill rules have been followed properly unless each element is considered individually.

- In complex control flows it is possible that a certain path can never be executed.
- In this case an improper define-use-kill combination might exist but will never be executed and so is not truly improper
- **For this reason, we now turn to dynamic data flow testing**

Dynamic Data Flow Testing

- Because data flow testing is based on a module's control flow, it assumes that the control flow is basically correct.
- The data flow testing process is to choose enough test cases so that:
  - Every "define" is traced to each of its "uses"
  - Every "use" is traced from its corresponding "define"
- To do this, enumerate the paths through the module

- This is done using the same approach as in control flow testing:
  - Begin at the module's entry point, take the leftmost path through the module to its exit. Return to the beginning and vary the first branching condition. Follow that path to the exit. ….
- for every variable, create at least one test case to cover every define-use pair.

Black Box Testing Techniques

- The general black box testing process is:
  - The requirements or specifications are analyzed.
  - Valid inputs are chosen based on the specification to determine that the software under test (SUT) processes them correctly.
  - Invalid inputs must also be chosen to verify that the SUT detects them and handles them properly.

- Expected outputs for those inputs are determined.
- Tests are constructed with the selected inputs.
- The tests are run.
- Actual outputs are compared with the expected outputs.
- A determination is made as to the proper functioning of the SUT

- Black box testing can be applied at all levels of system development—**unit**, **integration**, **system**, and **acceptance**.
- When using black box testing, the tester can never be sure of **how much of the system under test has been tested.**
- ***Even though we can't test everything, formal black box testing directs the tester to choose subsets of tests that are both efficient and effective in finding defects.***

Equivalence Class Testing

- Equivalence class testing is a technique used to reduce the number of test cases to a manageable level while still maintaining reasonable **test coverage**.
- An equivalence class consists of a set of data that is treated the same by the module or that should produce the same result.

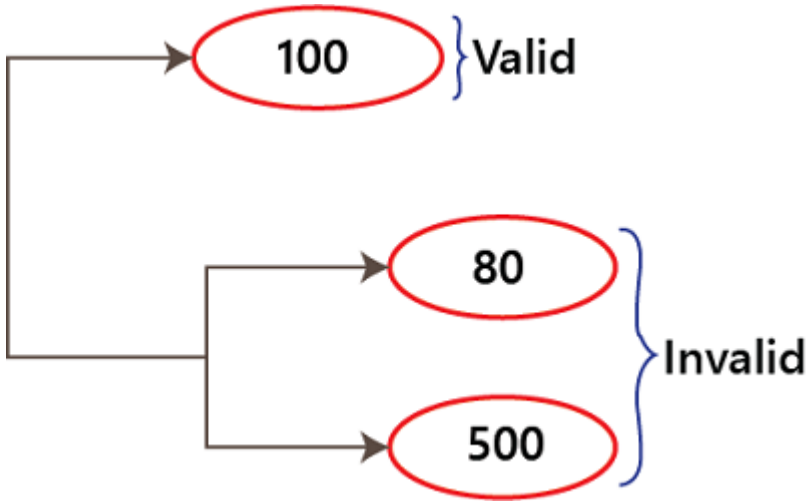
- A group of tests forms an equivalence class if you believe that:
  - They all test the same thing.
  - If one test catches a bug, the others probably will too.
  - If one test doesn't catch a bug, the others probably won't either

- We can decide the equivalent classes using the following two designing logic.
  - **Testing-by-contract :** Creates test cases only for the situations in which the pre-conditions are met.
    - modules are defined in terms of pre-conditions and post-conditions.
    - Post-conditions define what a module promises to do (compute a value, open a file, print report, update a database record, change the state of the system, etc.).
    - Pre-conditions define what that module requires so that it can meet its post-conditions.

- **Defensive testing:** an approach that tests under both normal and abnormal pre-conditions
  - If the normal preconditions are met, the module will achieve its normal post-conditions.
  - If the normal pre-conditions are not met, the module will notify the caller by returning an error code or throwing an exception

Example

- the Amount of test field accepts a Range (100-400) of values



- User ID—eight characters at least two of which are not alphabetic (numeric, special, nonprinting).

- Valid Classes
  - Length ≥ 8 AND ≥ 2 non-alphabetic characters
    - Example: ab12cd$e
- Invalid Classes
  - Length < 8 (even if other conditions are correct)
    - Example: ab1$cd
  - Length ≥ 8 but < 2 non-alphabetic characters
    - Example: abcdefg1 (only 1 non-letter)
  - Length ≥ 8 but all alphabetic characters
    - Example: abcdefgh

- Student ID—eight characters. The first two represent the student's home campus while the last six are a unique six-digit number. Valid home campus abbreviations are: AN, LC, RW, SM, TA, WE, and WN.

- **Valid Classes**
  - **Valid campus code + 6-digit number**
    - Example: AN123456

- **Invalid Classes**
  - **Invalid campus code**
    - Example: XX123456
  - **Correct campus code but last 6 not numeric**
    - Example: AN12A456
  - **Length < 8**
    - Example: AN12345
  - **Length > 8**
    - Example: AN1234567
  - **Correct format but digits not exactly 6**
    - Example: AN12345A

Exercise: Online Course Registration

- Rules:
  - Number of courses allowed: 1 to 5
  - Each course code must be non-empty string
- Define equivalence classes
- Create test cases.

- Equivalence Classes
  - **Based on Number of Courses**
    - **Valid Class:**
      - 1 ≤ number of courses ≤ 5
    - **Invalid Classes:**
      - 0 courses (less than minimum)
      - More than 5 courses
  - **Based on Course Code**
    - **Valid Class:**
      - Course code is a **non-empty string**
    - **Invalid Classes:**
      - Empty string ("")
      - (Optional depending on system) non-string input (e.g., numbers only)

- Test Cases
  - Valid Test Cases
    - 1 course with valid code
      - Input: ["CS101"]
      - Expected: Valid
    - 5 courses with valid codes
      - Input: ["CS101", "SE202", "MA303", "PH404", "EE505"]
      - Expected: Valid
  - Invalid Test Cases
    - 0 courses
      - Input: []
      - Expected: Invalid
    - More than 5 courses
      - Input: ["C1", "C2", "C3", "C4", "C5", "C6"]
      - Expected: Invalid
    - Empty course code
      - Input: ["CS101", ""]
      - Expected: Invalid
    - Null course code
      - Input: ["CS101", null]
      - Expected: Invalid
    - All invalid codes
      - Input: ["", "", ""]
      - Expected: Invalid

Boundary Value Testing

- Equivalence class testing is the most basic test design technique.
- It helps testers choose a small subset of possible test cases while maintaining reasonable coverage.
- Equivalence class testing has a second benefit. It leads us to the idea of boundary value testing

- Boundary value testing focuses on the boundaries because that is where so many defects hide.

```
If (applicantAge >= 0 && applicantAge <=16)
hireStatus="NO";
If (applicantAge >= 16 && applicantAge <=18)
hireStatus="PART";
If (applicantAge >= 18 && applicantAge <=55)
hireStatus="FULL";
If (applicantAge >= 55 && applicantAge <=99)
hireStatus="NO";
```

The interesting values on or near the boundaries in this example are
{-1, 0, 1}, {15, 16, 17},
{17, 18, 19}, {54, 55, 56}, and {98, 99, 100}.
Other values, such as {-42, 1001, FRED, %$#@} might be included depending on the module's documented preconditions.

- **Steps**
  - First, identify the equivalence classes.
  - Second, identify the boundaries of each equivalence class.
  - Third, create test cases for each boundary value by choosing one point on the boundary, one point just below the boundary, and one point just above the boundary.

Decision Table Testing

- Decision tables are an excellent tool to capture certain kinds of system requirements and to document internal system design.
- They are used to record complex business rules that a system must implement.
- In addition, they can serve as a guide to creating test cases.

	Rule 1	Rule 2	…….	Rule p
Conditions				
Condition 1				
Condition 2				
…..				
Condition m				
Action				
Action 1				
Action 2				
…				
Action n				

The general form of a decision table.

- An auto insurance company gives discounts to drivers who are married and/or good students.
- Let's begin with the conditions.
- The following decision table has two conditions, each one of which takes on the values Yes or No.

	Rule 1	Rule 2	Rule 3	Rule 4
Conditions				
Married?	Yes	Yes	No	No
Good Student?	Yes	No	Yes	No

	Rule 1	Rule 2	Rule 3	Rule 4
Conditions				
Married?	Yes	Yes	No	No
Good Student?	Yes	No	Yes	No
Actions				
Discount ($)	60	25	50	0

- Decision tables may specify more than one action for each rule.
- Again, these rules may be unique or may be shared.

	Rule 1	Rule 2	Rule 3	Rule 4
Conditions				
Condition-1	0–1	1–10	10–100	100–1000
Condition-2	<5	5	6 or 7	>7
Actions				
Action-1	Do X	Do Y	Do X	Do Z
Action-2	Do A	Do B	Do B	Do B

- Each rule (vertical column) becomes a test case but values satisfying the conditions must be chosen.

Test Case ID	Condition-1	Condition-2	Expected Result
TC1	0	3	Do X / Do A
TC2	5	5	Do Y / Do B
TC3	50	7	Do X / Do B
TC4	500	10	Do Z / Do B

Applicability and Limitations

- Decision Table testing can be used whenever the system must:
  - implement complex business rules
  - when these rules can be represented as a combination of conditions and
  - when these conditions have discrete actions associated with them

Create a DT and test cases for the following scenario.

A bank has developed an automated system to process loan applications. The system evaluates each applicant based on three criteria:

- The applicant's **monthly income**
- The applicant's **credit score**
- The applicant's **existing debt level**

The system follows these rules:

- If the applicant has a **monthly income of at least $3000**, a **credit score of 700 or above**, and **low existing debt**, the loan is **approved automatically**.
- If some of the conditions are satisfied but not all, the application is **sent for manual review**.
- If the applicant has **low income**, or **poor credit score**, or **high debt**, and does not meet sufficient criteria, the loan is **rejected**.

Conditions	R1	R2	R3	R4	R5	R6	R7	R8
Income ≥ 3000	Y	Y	Y	Y	N	N	N	N
Credit score ≥ 700	Y	Y	N	N	Y	Y	N	N
Debt low	Y	N	Y	N	Y	N	Y	N
----------	----	----	----	----	----	----	----	----
Approve Loan	Y	N	N	N	N	N	N	N
Manual Review	N	Y	Y	N	Y	N	N	N
Reject Loan	N	N	N	Y	N	Y	Y	Y

Use Case Testing

- Use cases comes with a description that explains them in detail.
- Each use case has been through an inspection process before it was implemented.
- To test the implementation, the basic rule is to create at least one test case for the main success scenario and at least one test case for each extension.
- Because use cases do not specify input data, the tester must select it.
- Typically, we use the equivalence class and boundary value techniques described earlier.

Example

Use Case Component	Description
Use Case Number or Identifier	SURS1138
Use Case Name	Register for a course (a class taught by a faculty member)
Goal in Context	
Scope	System
Level	Primary task
Primary Actor	Student
Preconditions	None

Success End Conditions	The student is registered for the course—the course has been added to the student's course list	
Failed End Conditions	The student's course list is unchanged	
Trigger	Student selects a course and "Registers"	
Main Success Scenario A: Actor S: System	**Step**	**Action**
	1	A: Selects "Register for a course"
	2	A: Selects course (e.g. Math 1060)
	3	S: Displays course description
	4	A: Selects section (Mon & Wed 9:00am)
	5	S: Displays section days and times
	6	A: Accepts
	7	S: Adds course/section to student's course list

Extensions	2a	Course does not exist S: Display message and exit
	4a	Section does not exist S: Display message and exit
	4b	Section is full S: Display message and exit
	6a	Student does not accept S: Display message and exit

Pairwise testing with orthogonal array

- It is software testing technique that uses orthogonal arrays to create test cases.
- It is statistical testing approach especially useful when system to be tested has huge data inputs.
- Orthogonal array testing helps to maximize test coverage by pairing and combining the inputs and testing the system with comparatively a smaller number of test cases for time saving.

- **Orthogonal array:**
  - A table of test values where, if you look at any two columns, every possible pairing of their values shows up the same number of times.
  - That balance is what lets a small number of rows still cover every combination of every pair of inputs, which is the whole point in testing: you get full pairwise coverage without needing every possible combination of all inputs at once.

Example

- A Web page has three distinct sections (Top, Middle, Bottom) that can be individually shown or hidden from a user
  - No of Factors = 3 (Top, Middle, Bottom)
  - No of Levels (Visibility) = 2 (Hidden or Shown)

- If we go for Conventional testing technique, we need test cases like 2^3 = 8 Test Cases

Top	Middle	Bottom
H	H	H
H	H	S
H	S	H
H	S	S
S	H	H
S	H	S
S	S	H
S	S	S

- **L4 (2-level, small problems)**
- Used when:
  - 2 levels (e.g., Yes/No)
  - Up to ~3 factors

Test	A	B	C
1	1	1	1
2	1	2	2
3	2	1	2
4	2	2	1

- If we go for OAT testing, we need 4 Test cases as shown below:

Test Cases	TOP	Middle	Bottom
Test #1	Hidden	Hidden	Hidden
Test #2	Hidden	Visible	Visible
Test #3	Visible	Hidden	Visible
Test #4	Visible	Visible	Hidden

Reading Assignment
L8, L9, and L16

Exercise

- **Scenario:** Testing a Mobile App Login Function
- You want to test the login feature of a mobile app. There are 3 factors (inputs) affecting login success:

Factor	**Levels**
A: Device Type	Android, iOS
B: Network Type	WiFi, Mobile Data
C: User Type	New User, Existing User

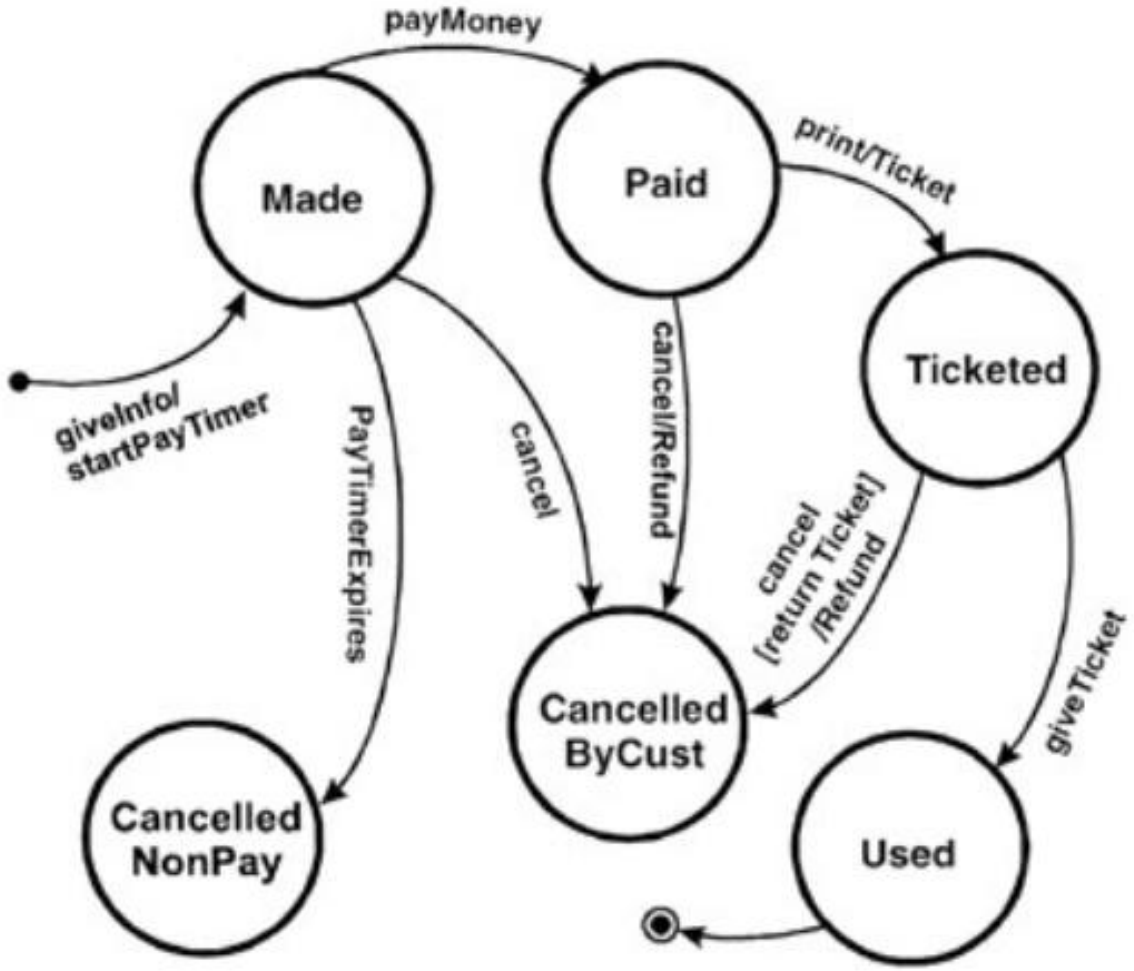
- Use orthogonal array to create the test cases

Device Type	Network Type	User Type
Android	WiFi	New User
Android	Mobile Data	Existing User
iOS	WiFi	Existing User
iOS	Mobile Data	New User

State-Transition Testing

- State-Transition diagrams, like decision tables, are another excellent tool to capture certain types of system requirements and to document internal system design.
- These diagrams document the events that come into and are processed by a system as well as the system's responses.
- These diagrams are also vital tools in the tester's personal toolbox

Example of state transition diagram (ticket Reservation)

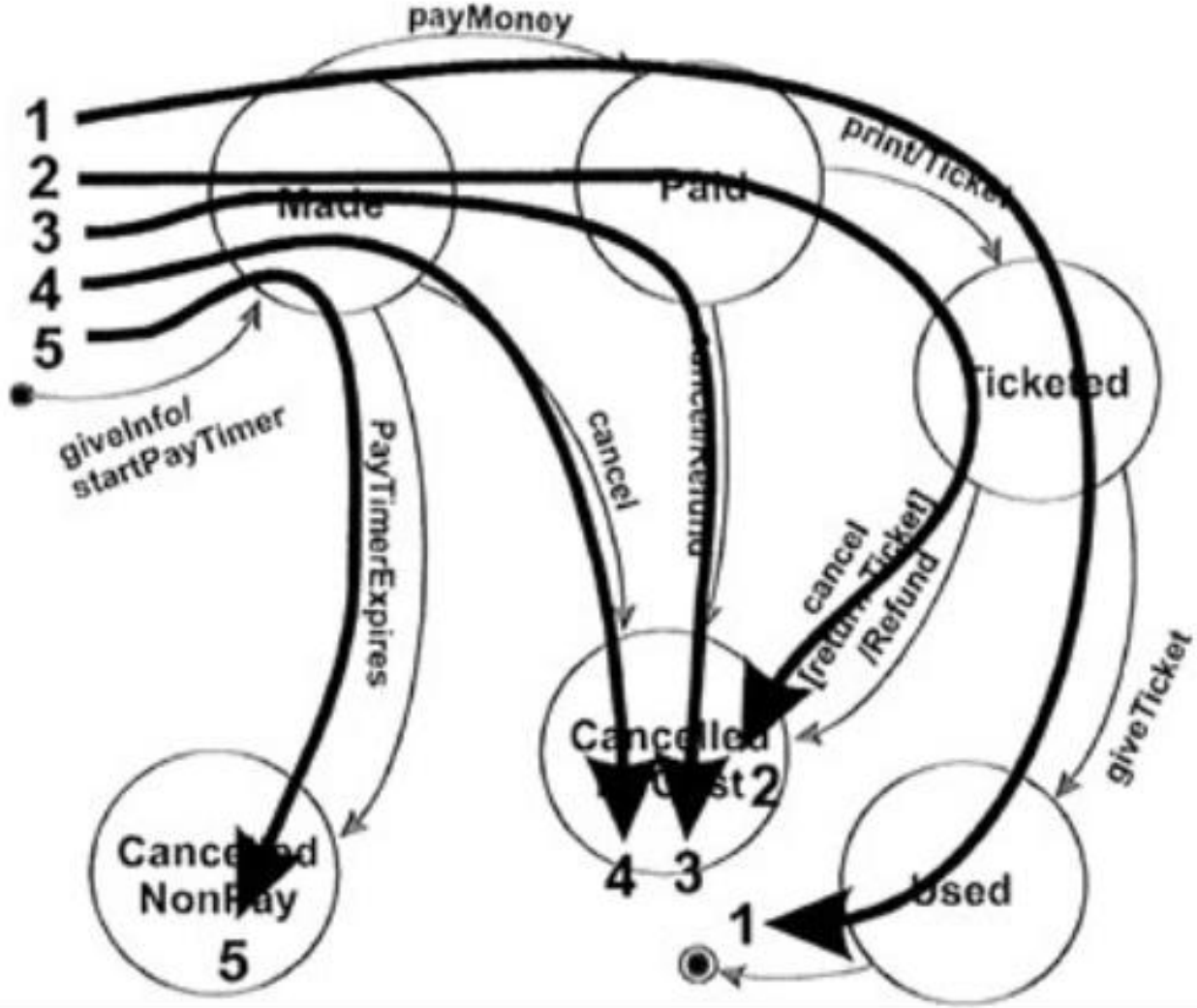


State-Transition Tables

Current State	Event	Action	Next State
null	giveInfo	startPayTimer	Made
null	payMoney	--	null
null	print	--	null
null	giveTicket	--	null
null	cancel	--	null
null	PayTimerExpires	--	null
Made	giveInfo	--	Made
Made	payMoney	--	Paid

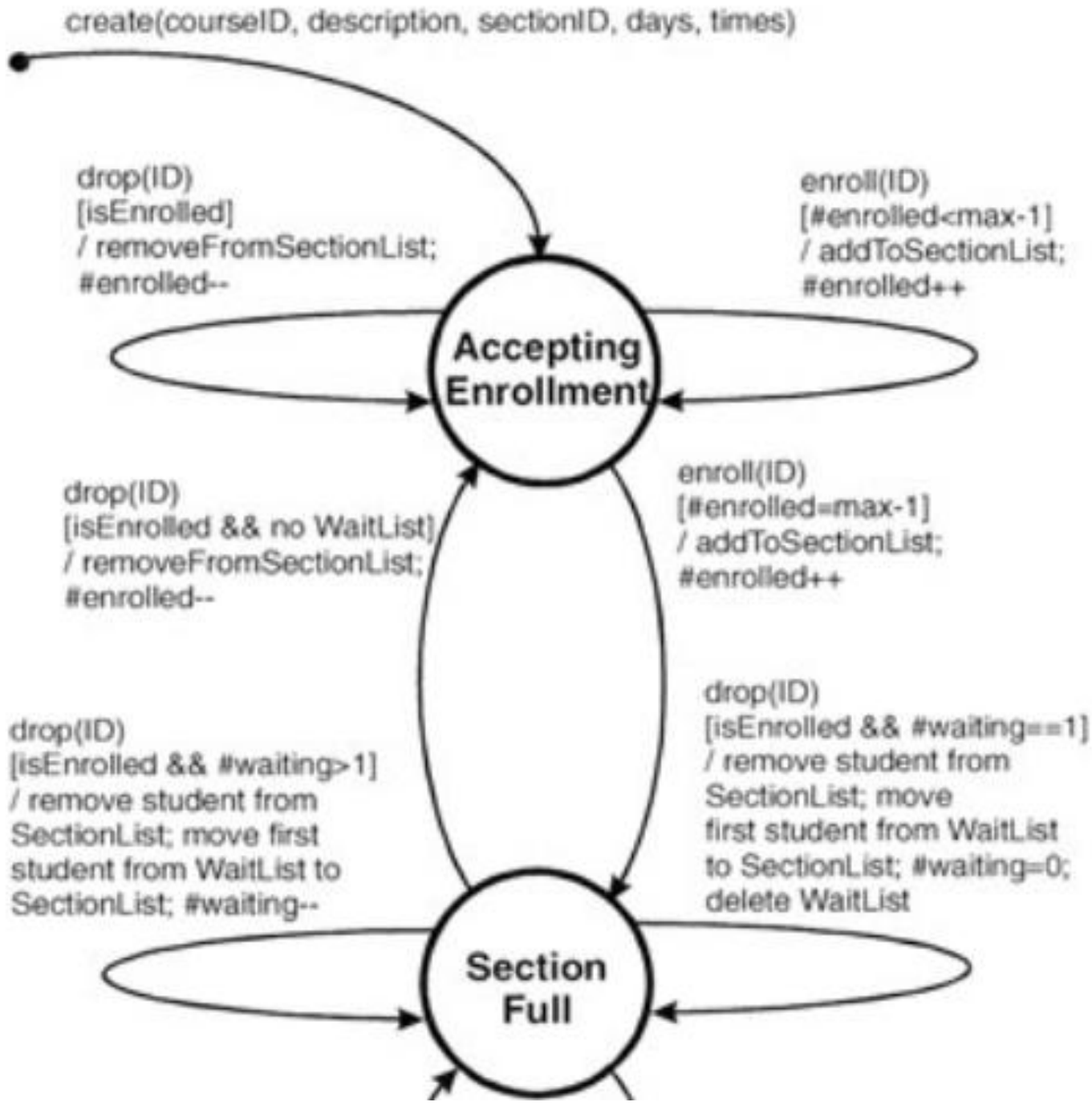
Made	print	--	Made
Made	giveTicket	--	Made
Made	cancel	--	Can-Cust
Made	PayTimerExpires	--	Can-NonPay
Paid	giveInfo	--	Paid
Paid	payMoney	--	Paid
Paid	print	Ticket	Ticketed
Paid	giveTicket	--	Paid
Paid	cancel	Refund	Can-Cust
Paid	PayTimerExpires	--	Paid

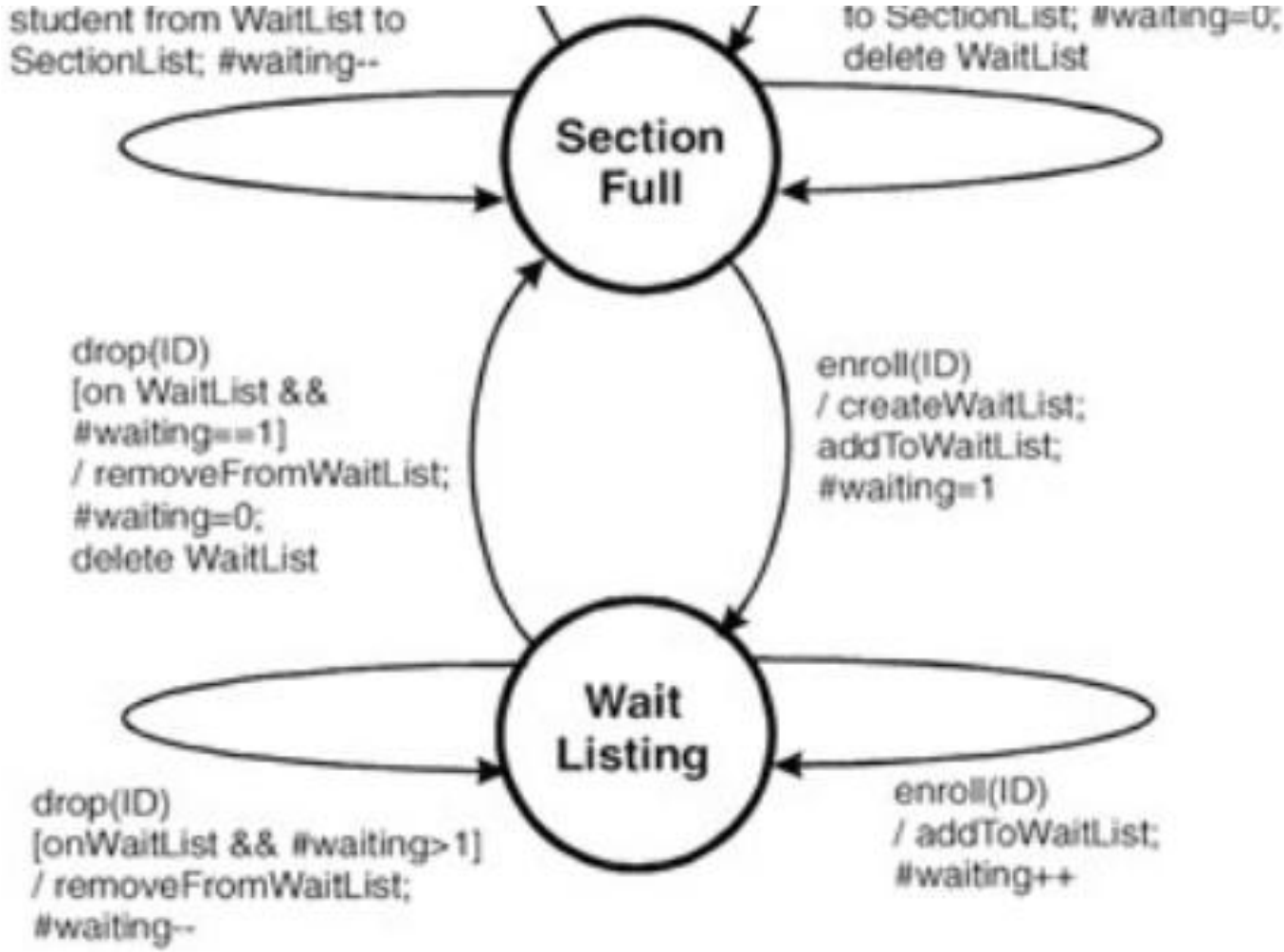
- Create a set of test cases such that all transitions are exercised at least once under test.
- This level of testing provides a good level of coverage without generating large numbers of tests.
- This level is generally the one recommended


payMoney
1
2
3
4
5
Made
Paid
printTicket
Ticketed
giveInfo/
startPayTimer
PayTimerExpires
cancel
cancel
/Refund
giveTicket
Cancelled
NonPay
5
4
3
1

Example

- Below is a state-transition diagram for the "enroll in a course" and "drop a course" process. Determine a set of test cases that you feel adequately cover the enroll and drop process.


create(courseID, description, sectionID, days, times)
drop(ID)
[isEnrolled]
/ removeFromSectionList;
#enrolled--
enroll(ID)
[#enrolled<max-1]
/ addToSectionList;
#enrolled++
Accepting Enrollment
drop(ID)
[isEnrolled && no WaitList]
/ removeFromSectionList;
#enrolled--
enroll(ID)
[#enrolled=max-1]
/ addToSectionList;
#enrolled++
drop(ID)
[isEnrolled && #waiting>1]
/ remove student from SectionList; move first student from WaitList to SectionList; #waiting--
drop(ID)
[isEnrolled && #waiting==1]
/ remove student from SectionList; move first student from WaitList to SectionList; #waiting=0; delete WaitList
Section Full


student from WaitList to SectionList; #waiting--
to SectionList; #waiting=0; delete WaitList
Section Full
drop(ID) [on WaitList && #waiting==1] / removeFromWaitList; #waiting=0; delete WaitList
enroll(ID) / createWaitList; addToWaitList; #waiting=1
Wait Listing
drop(ID) [onWaitList && #waiting>1] / removeFromWaitList; #waiting--
enroll(ID) / addToWaitList; #waiting++

Reading Assignment
Domain Analysis Testing